Assignment: Data Structures & Systems Design (SE Intern) Overview

This assessment evaluates your proficiency in fundamental data structures, algorithm design, and your ability to explain complex logic in plain English. You are required to solve two distinct problems.

Note: For each problem, please provide a brief explanation of your logic and approach in your own words before the code block.

Problem 1: **LRU Cache Implementation**

Implement a Least Recently Used (LRU) Cache with the following operations:
get(key): Returns the value associated with the key if it exists, otherwise returns -1. This operation must mark the key as recently used.
put (key, value): Adds or updates the key-value pair. If the cache is at capacity, it must evict the least recently used item before adding the new one.
Requirements:
Both get and put operations must run in $O(1)$ time complexity.
The cache has a fixed capacity set during initialization.
In your own words: Explain why you chose your specific data structures to achieve $O(1)$ performance.

Problem 2: Event Scheduler

You are designing a scheduling system for a meeting platform. Given a list of events represented as (start time, end time) tuples, implement the following:
can_attend_all(events): Returns True if a person can attend all events without overlaps, False otherwise.
min_rooms_required(events): Returns the minimum number of meeting rooms required to schedule all events simultaneously.
Constraints:
Input format: [(9, 10), (10, 11), (11, 12)].
Adjacent events (where an end time equals a start time) are not considered overlaps.
In your own words: Explain the algorithmic logic behind your room requirement calculation.

Final Discussion & Analysis
Please include a response to the following:
Time & Space Complexity: Provide an analysis for all functions implemented in both problems.
Trade-offs: Why are a combination of a hash map and a doubly linked list often used for an LRU cache?
Future Proofing: How would you modify your scheduler to assign specific room numbers (e.g., "Room A", "Room B") to each event?
Concurrency: What changes would be required to make the LRU Cache thread-safe?